Endpoint configuration

This guide helps you configure and manage API endpoints to enable secure integrations between your system and third-party applications. You'll learn how to set up endpoints, choose authentication methods, configure security, and troubleshoot common issues.

Target Audience

- Solution Architects
- System Administrators
- Technical Administrators
- DevOps Engineers
- Security Engineers
- Technical Leads

What You'll Learn

- What endpoints are and why they matter
- How to configure endpoints step-by-step
- Different authentication methods and when to use them
- Security features to protect your data
- How to use filtersets for fine-grained control
- Best practices for secure integrations
- How to resolve common issues

Contents

Understanding API Endpoints
- What is an Endpoint?
- Why Configure Endpoints?

Step-by-Step Configuration Guide
- Step 1: Choose the Integration Type
- Step 2: Select Authentication Method
- Step 3: Set Permissions
- Step 4: Configure Client ID (Two-Factor Authentication)
- Step 5: Set Request Limits (Rate Limiting)
- Step 6: Create Service User (For AuthKey Authentication)
- Step 7: Advanced Filtering (applies to SIF RPC)
- Step 8: CORS Configuration (For Web-Based Integrations)

Authentication Methods Explained
- What Gets Authenticated?
- Detailed Authentication Modes
  - 1. Bearer Token (OAuth 2.0 / JWT)
  - 2. AuthKey
  - 3. Combined Authentication (JWT + AuthKey) (Combined user)
  - 4. Client Credential Grant (Combined daemon) + AuthKey

- Authentication Comparison Table

Security Features
- 1. API Keys (AuthKey)
- 2. Client ID
- 3. Rate Limiting
- 4. CORS (Cross-Origin Resource Sharing)
- 5. Content Validation & Permissions
- 6. Service Delegate Secret

Advanced Features (applicable only for SIF RPC integration point)
- What Are Filtersets?
- How Filtersets Work
- Configuring Filtersets
- Benefits of Using Filtersets
  - 1. Enhanced Security
  - 2. Improved Data Privacy
  - 3. Better Performance
  - 4. Fine-Grained Control
  - 5. Simplified Maintenance
- Filterset Configuration Example
- Best Practices for Filtersets

Best Practices
- Authentication Best Practices
  - ✅ DO
  - ❌ DON'T
- Security Best Practices
  - Permission Management
  - Rate Limiting
  - Client ID Best Practices
- Configuration Management Best Practices
  - Organization
- Maintenance Best Practices
  - Key Rotation
- Quick Security Checklist

Troubleshooting
- Common Issues and Solutions
  - Issue 1: 401 Unauthorized Error
  - Issue 2: 403 Forbidden Error
  - Issue 3: 404 Not Found Error
  - Issue 4: CORS Errors (Browser Only)
  - Issue 5: Configuration Changes Not Applied
  - Issue 6: Token Validation Fails
- Diagnostic Tips
  - Test Authentication
  - Get Help Information
- Error Code Quick Reference

Summary

- Key Takeaways
  - 1. Choose the Right Authentication
  - 2. Layer Your Security
  - 3. Configure Properly
  - 4. Maintain Security
  - 5. Monitor and Improve
- Quick Reference Cards
  - Authentication Methods Quick Guide
  - Security Quick Guide
  - Troubleshooting Quick Guide

Additional Resources
- Getting Help
- Best Practice Templates

Miscellaneous
- How an endpoint is selected when a Bearer token is sent together with an AuthKey and ClientID

Changelog

Understanding API Endpoints

What is an Endpoint?

An **endpoint** is a specific address (URL) where your application or integration connects to exchange data or perform actions. Think of it as a doorway between your system and another, allowing secure and controlled communication.

An endpoint configuration includes:

- **URL Pattern:** The specific web address (e.g., `/v2/api/customers/*`)
- **Authentication Method:** How users or systems prove their identity
- **Permissions:** What actions are allowed (Read, Write, Delete, Update)
- **Security Controls:** Rate limits, access keys, CORS settings
- **Service Filters:** Which services and methods can be accessed

Why Configure Endpoints?

Benefit	Description
Security	Ensures only authorized users or systems can access your data
Control	Lets you decide what data and actions are available to each integration
Audit & Logging	Tracks who did what, helping with troubleshooting and compliance
Performance	Prevents system overload through rate limiting
Privacy	Restricts data exposure to only what's necessary

Step-by-Step Configuration Guide

Step 1: Choose the Integration Type

When creating a new endpoint, you'll need to identify:

- **Third Party Application:** Give your integration a descriptive name for easy identification
  - Example: "Customer Portal API", "Mobile App Integration", "Reporting Service"
- **Integration Point:** Select where and how your system will connect. Choose integration point from the list of Public360 integrations below.
  - **Available Public360 integration points (select the appropriate one):**
    - Noark4
    - GI Innsyn
    - GI Oppdatering
    - GI Sakfaser
    - GI eArchive Innsyn
    - GI eArchive Oppdatering
    - GI eArchive Sakfaser
    - GI Offentlig Innsyn
    - GI Offentlig Sakfaser
    - GI Unregistered document
    - EDUImport
    - OEPExporter
    - Casehandler
    - Public citizen
    - Public file
    - Package upload
    - SIF RPC
    - Board handling API
    - eService
    - eMeetings prepare RPC
    - eMeetings prepare
    - OpenGov meetings
    - OpenGov gateway

⚠️ Available integrations may vary per customer because some are installed only during module installation

- **Guidance:**
  - Pick the single integration point that matches the primary responsibility for this endpoint.
  - If an integration must span multiple responsibilities, select multiple points and identify and map which services/methods each will use (relevant for SIF RPC).
  - If you don't see the required integration in this list, confirm the customer's installed modules or contact the administrator to enable the missing integration.

Step 2: Select Authentication Method

Authentication is how you prove who you are. Choose the method that best fits your use case:

Primary consideration — grant only the permissions required (Principle of Least Privilege). Before choosing an auth mode, decide which services/methods (SIF RPC) the integration must call and which data fields it must see.

Currently supported authentication modes:

- AuthKey
- Bearer
- Combined user
- Combined daemon

Short descriptions:

- **AuthKey** — Service-to-service using a dedicated service user and API key. Best for automated integrations.
  - Examples:
    - Scheduled batch jobs / nightly data syncs
    - Background workers / ETL pipelines
    - Document import/export services or file processors
    - Webhook consumers that process external events
    - CI/CD automation / deployment hooks calling internal APIs
- **Bearer** — JWT tokens representing a user. Best for user-driven integrations (web/mobile).
  - Examples:
    - Single-page applications (SPAs) where users interactively view and modify their data (customer portals, dashboards)
    - Native mobile apps (iOS/Android) using SSO and calling APIs on behalf of the signed-in user
    - Web-based admin consoles and support tools that require actions to be attributed to a specific user
    - Delegated third-party apps (OAuth apps) acting on behalf of users
    - Browser extensions or client-side integrations that perform user-initiated requests
- **Combined user** — Requires both a valid user JWT and an AuthKey (user context + integration key).
- **Combined daemon** — Combined mode for daemon/daemon-like integrations (client credentials / app token + AuthKey) — no interactive user context.
  - A "daemon" is simply a background program or service that runs without a person actively using it — like a robot running on a server.
  - Examples: nightly data import jobs, scheduled report generators, or a service that polls an external system and processes events.
  - Combined daemon mode is how these background robots authenticate: they use an application token (client credentials) plus an AuthKey so the system can identify the app and apply the correct permissions.

Key considerations (in priority order):

- Principle of Least Privilege: map required API methods + result fields → apply filtersets (apply to SIF RPC) and minimal roles instead of broad permissions.
- Auth type that matches the actor:
  - User-driven integrations → Bearer (JWT)

    - Service-to-service → AuthKey (dedicated service user) or Client Credential grant
    - High-sensitivity → Combined (JWT + AuthKey) ± ClientID ± ServiceDelegateSecret
- Role & group assignment: ensure service users or client accounts are assigned to the correct roles/groups to enforce least privilege.
- Audit & traceability: choose auth that supports meaningful logging (tokens with user claims or unique service users) so actions can be traced to an identity.
- Token/key lifecycle and rotation: ensure keys and tokens have an expiration and a defined rotation cadence before rollout. This can be done by setting the endpoint to inactive and configuring new one.
- Transport & CORS: require HTTPS; configure CORS only when browser access is needed.
- Rate limiting & error handling: pick auth that allows per-integration rate limits and test 429 handling.
    - **What happens when limit is exceeded:**
        - The API returns a "429 Too Many Requests" error
        - The caller must wait before trying again
        - Actions are logged for monitoring

- - About HTTP 429 (Too Many Requests)
        - Meaning: HTTP 429 indicates the client has exceeded the allowed number of requests within the configured time window. The server may include a Retry-After header (seconds or timestamp) or a response body with guidance (e.g., {"error":"Rate limit exceeded","message":"Maximum 100 requests per minute allowed","retryAfter":45}).
        - Recommended client behavior: stop sending requests, respect the Retry-After value if present, and implement exponential backoff before retrying.

- - How to test 429 locally
        - Run a short loop to hammer a test endpoint until you see 429:
        - Look for "429" in the output and inspect response headers:
        - Tips:
            - Test against a development environment or an endpoint with a deliberately low rate limit to avoid impacting production.
            - Check the Retry-After header or response body to know when it's safe to retry.
- Test and rollback: validate permissions in a dev environment, verify filtersets, then deploy.

Notes:

- Adjustments to roles, filtersets and group membership can be done in the admin portal, please contact Public360 support for assistance.
- If unsure, prefer a restrictive AuthKey/service-user + filterset configuration for automated integrations and add JWT or ClientID factors only where user-context or higher assurance is required.

Authentication Type	Best For	Security Level
Bearer Token (JWT)	User-based access, web/mobile apps	High
AuthKey	Service-to-service, automated integrations	Medium-High
Combined (JWT + AuthKey)	High-security scenarios	Very High
Client Credential Grant	Applications acting independently	High

Step 3: Set Permissions

Permissions define what the integration can do:

Permission	Description	Example Use Case
Read	Only view data	Reporting dashboard
Write	Create or edit data	Form submission integration

Important Notes:

- For **Bearer (JWT)** authentication: Permissions follow the user's rights in the Public 360
- For **AuthKey** authentication: You set permissions explicitly for the service user

Step 4: Configure Client ID (Two-Factor Authentication)

A **Client ID** uniquely identifies the application making requests and acts as an extra layer of security.

When to use:

- ✅ High-security integrations
- ✅ External third-party applications

How it works:

- Client ID is sent in request headers (not URL for security)
- Acts as a second authentication factor alongside JWT or AuthKey

Step 5: Set Request Limits (Rate Limiting)

To keep the system running smoothly, set a limit on how many requests an integration can make per minute.

Typical Settings:

- Public APIs: 100 requests/minute
- Internal integrations: 200-500 requests/minute
- High-volume integrations: Custom limits based on needs

What happens when limit is exceeded:

- The API returns a "429 Too Many Requests" error

- The caller must wait before trying again
- Actions are logged for monitoring

Step 6: Create Service User (For AuthKey Authentication)

When using AuthKey authentication, create a dedicated service user for each integration.

Why create separate service users:

- ✅ Track actions per integration
- ✅ Isolate permissions and access
- ✅ Easier troubleshooting
- ✅ Better security (if one is compromised, others remain safe)

Best Practices:

- Create a new service user for each integration
- Never share service users between integrations
- Use descriptive names (e.g., "SvcUser_CustomerPortal", "SvcUser_ReportingAPI")
- Maintain a record of service user assignments in your organization's configuration management system (e.g., internal wiki, solution description, integration document) noting which service user is associated with each integration

Step 7: Advanced Filtering (applies to SIF RPC)

Restrict integrations to only certain services or methods for improved security and performance.

Service Name & Method Name:

- Limit each endpoint to only the services needed
- Example: Only allow "GetCases" if the integration just needs case data

Permission Sets (Filtersets):

- Use templates to restrict what data is returned
- Example: Only return case number, title, and date—hide sensitive fields

See Advanced Features (applicable only for SIF RPC integration point) for detailed information on filtersets.

Step 8: CORS Configuration (For Web-Based Integrations)

Note: The Teams integration endpoint (`/v2/api/call/SI.Client.Api.Custom.Teams`) and the OWA endpoint (`/v2/api/call/SI.Biz.Core.OfficeAddin`) are built-in product endpoints. CORS for these is set in AuthProxy `App.config` (`Teams.CORS.*` / `OWA.CORS.*`), not via this UI.

CORS (Cross-Origin Resource Sharing) controls which websites or apps can access your API.

Configuration Elements:

Setting	Description	Example
Allowed Origins	Which websites can access the API	`https://app.dummycompany.com`
Allowed Methods	Which HTTP methods are permitted	GET, POST, OPTIONS
Allowed Headers	Which headers can be sent	authorization, content-type
Allow Credentials	Whether cookies/tokens are allowed	true/false
Preflight Max Age	How long to cache CORS settings	600 seconds (10 minutes)

When to configure CORS:

- ✅ Web-based integrations (browser apps)
- ✅ Single-page applications (SPAs)
- ✅ JavaScript-based integrations

Authentication Methods Explained

What Gets Authenticated?

When an API call is made, the system checks:

1. **Who is making the request?** (User, app, or service)
2. **How are they proving their identity?** (Token, key, or credentials)
3. **What are they allowed to do?** (Permissions check)

Detailed Authentication Modes

1. Bearer Token (OAuth 2.0 / JWT)

How it works:

- The caller presents a token (JWT) obtained from a trusted identity provider
- The API validates the token against the identity provider
- User information is extracted from the token (email, username)
- Permissions are applied based on the user's rights in the Public 360

Who gets authenticated:

- The user or application that owns the token

Best for:

- User-based access
- Mobile apps
- Web applications
- Integrations needing granular, user-level permissions

Authentication Flow:

User/App → Identity Provider (Azure AD) → Receives Token User/App → API (with token) → API validates token → Access granted

Request Example:

```
GET /v2/api/customers HTTP/1.1 Host: api.company.com Authorization: Bearer
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

2. AuthKey

How it works:

- The caller provides a secret key assigned to a specific service user
- The API checks the AuthKey and validates it
- Actions are performed as the service user associated with the key

Who gets authenticated:

- The integration or service user associated with the AuthKey

Best for:

- Automated integrations
- Service-to-service communication
- When a dedicated service user is needed
- Background jobs and scheduled tasks

Request Example:

```
GET /v2/api/customers HTTP/1.1 Host: api.company.com AuthKey: 8256344C-36DF-414D-B7A4-8ADFA5F5C1E6
```

Security Tips:

- Generate a unique AuthKey for each integration
- Store AuthKeys securely (Azure Key Vault, environment variables)
- Never hardcode AuthKeys in application code
- Rotate AuthKeys regularly

3. Combined Authentication (JWT + AuthKey) (Combined user)

How it works:

- Requires both a Bearer token and an AuthKey
- Both credentials must be valid for access

Who gets authenticated:

- Both the user/application (via token) and the integration (via AuthKey)

Best for:

- High-security scenarios

- Separation of duties
- External partner integrations

Request Example:

```
POST /v2/api/sensitive-data HTTP/1.1 Host: api.company.com Authorization: Bearer
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9... AuthKey: SECRET-KEY-HERE ClientID: partner-app-id
```

4. Client Credential Grant (Combined daemon) + AuthKey

Client credential grant requires additional information (such as AuthKey and ClientID) to identify different clients, as described here: https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow

This authentication mode is used when an application acts independently without an interactive user session (daemon/background service). The challenge with client credential grants is that no user identity is attached to the token. To address this, the system maps the application token to a specific Public 360 service user through the endpoint configuration using the provided AuthKey.

How it works:

- Used by applications (not users) to authenticate themselves
- Application provides a token and authkey
- The API validates credentials and links to a service user

Who gets authenticated:

- The application itself

Best for:

- Background services
- Daemons and scheduled jobs
- Automated integrations without user context

Required headers for each SIF request:

- `Authorization: Bearer <token>` — the "Bearer " prefix is mandatory
- `authkey: <value from endpoint configuration in 360 admin>` — always required
- `ClientID: <app registration Client ID>` — required if Client ID is configured on the endpoint

OAuth scope for token request:

The scope MUST target the 360 app registration, not Microsoft Graph.

- Correct: `api://<client-id>/.default`
- Wrong: `https://graph.microsoft.com/.default`

A token requested with a Graph scope is signed with different keys and will fail with IDX10511 (Signature validation failed). Many Azure tutorials default to `graph.microsoft.com` — this does not work for SIF.

Request Example:

```
POST /Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/CaseService/GetCases HTTP/1.1
Host: your-environment.public360online.com
Authorization: Bearer <app-token>
authkey: <authkey-from-endpoint-config>
ClientID: <app-client-id>
```

Authentication Comparison Table

Authentication Method	Security Level	User Context	Best Use Case	Session Type
Bearer Token (JWT)	High	Dynamic (token claims)	Web/mobile apps, user-based access	Stateless
AuthKey	Medium-High	Fixed (service user)	Service-to-service, automation	None
Combined (JWT+AuthKey)	Very High	Both dynamic & fixed	High-security integrations	Stateless
Client Credential Grant	High	Application identity	Background services	Stateless

Data access scope per authentication mode:

- **Bearer (JWT):** Data access follows the authenticated user's access matrix
- **AuthKey:** Service user has full data access (role 24) — access matrix does not apply
- **Combined User (JWT + AuthKey):** Data access follows the authenticated user's access matrix
- **Combined Daemon (Client Credential Grant):** Mapped service user has full data access (role 24) — access matrix does not apply
- To restrict data access for service-to-service integrations, create a regular 360° user instead of a service user (see Integration users)

Security Features

1. API Keys (AuthKey)

Purpose: Provides an additional security layer beyond basic authentication.

How to send AuthKey:

```
Method 1 - HTTP Header (Recommended): AuthKey: YOUR-KEY-HERE
Method 2 - Query Parameter (Not recommended for production): ?authkey=YOUR-KEY-HERE
```

Best Practices:

- ✅ Use random GUIDs (e.g., `8256344C-36DF-414D-B7A4-8ADFA5F5C1E6`)
- ✅ Generate a unique key for each endpoint
- ✅ Store keys securely (Azure Key Vault, environment variables)
- ✅ Rotate keys regularly (e.g., every 6 months)
- ✅ Use different keys for different environments (dev, test, prod)
- ❌ Never hardcode keys in application code

- ❌ Don't use simple/predictable keys like "key123"
- ❌ Don't share keys between integrations
- ❌ Don't send AuthKeys in URLs (use headers instead)

AuthKey Rotation Plan (Client guidance):

1. Generate new AuthKey
2. Update endpoint config with new key
3. Update clients to use new key

2. Client ID

Purpose: Uniquely identifies the application making requests and acts as a second authentication factor.

Benefits:

- Track which apps use your APIs
- Restrict access to specific applications
- Audit usage by application
- Prevent unauthorized apps from accessing endpoints

How to send Client ID:

```
Method 1 - HTTP Header (Recommended): ClientID: your-client-identifier
Method 2 - Query Parameter: ?ClientID=your-client-identifier
```

When to use:

- High-security integrations
- External third-party applications
- When you need app-level tracking and control

3. Rate Limiting

Purpose: Protects APIs from excessive requests and prevents system overload.

How it works:

- Tracks request timestamps in a rolling window
- Removes requests older than 1 minute
- Compares current request count against limit
- Returns error if limit exceeded

Configuration Examples:

API Type	Typical Limit	Reason
Public APIs	100/minute	Prevent abuse, fair usage
Internal integrations	200-500/minute	Higher trust, more capacity
High-volume integrations	Custom (1000+/minute)	Business critical, negotiated

Error Response:

```
HTTP 429 Too Many Requests
{
 "error": "Rate limit exceeded",
 "message": "Maximum 100 requests per minute allowed",
 "retryAfter": 45
}
```

Best Practices:

- Set realistic limits based on expected usage
- Monitor actual usage and adjust as needed
- Implement exponential backoff in client applications
- Log rate limit violations for abuse detection

4. CORS (Cross-Origin Resource Sharing)

Purpose: Controls which websites or web apps can access your API from browsers.

When CORS is needed:

- Web-based integrations (browser apps)
- Single-page applications (SPAs)
- JavaScript-based clients
- Any cross-domain browser requests

Configuration Elements:

Setting	Description	Example	Security Impact
Enable CORS	Turn CORS on/off	`true`	Must be enabled for browser access
Allowed Origins	Specific websites allowed	https://app.company.com	Prevents unauthorized domains
Wildcard Origins	Allow all domains	`*` (not recommended)	⚠️ Security risk—avoid in production
Allowed Methods	Permitted HTTP methods	`GET, POST, OPTIONS`	Restricts operations
Allowed Headers	Headers that can be sent	`authorization, content-type`	Controls what clients can send
Allow Credentials	Include cookies/auth	`true`	Required for authenticated requests
Preflight Max Age	Cache duration	`600` seconds	Improves performance

Security Best Practices:

- ✅ Always specify exact origins (avoid wildcards)
- ✅ Only allow necessary methods
- ✅ Limit allowed headers to minimum required
- ✅ Set appropriate preflight cache duration
- ❌ Never use `*` for Allowed Origins in production
- ❌ Don't allow credentials with wildcard origins

5. Content Validation & Permissions

Purpose: Restricts which operations can be performed through endpoints.

Service-Level Control:

- Limit endpoint to specific services (e.g., only CaseService)
- Restrict to specific methods (e.g., only GetCases)
- Use permission sets to control data access

Permission Types:

Permission	Description	Use Case
Read	View data only	Reporting, dashboards
Write	Create new data	Form submissions

Best Practices:

- Principle of Least Privilege: map required API methods + result fields → apply filtersets (apply to SIF RPC) and minimal roles instead of broad permissions.
- Grant minimum necessary permissions
- Use permission sets to restrict data fields
- Regularly audit permissions
- Document why each permission is needed
- Role & group assignment: ensure service users or client accounts are assigned to the correct roles/groups to enforce least privilege.

6. Service Delegate Secret

Purpose: Additional authentication factor for high-security scenarios.

How it works:

- Works alongside AuthKey and Client ID
- All three must be valid for access
- Provides triple-layer authentication

When to use:

- External partner integrations

- High-value or sensitive data access
- Regulatory compliance requirements

Validation Chain:

- AuthKey must match ✅
- ClientID must match ✅
- ServiceDelegateSecret must match ✅ → All three valid = Access granted

Advanced Features (applicable only for SIF RPC integration point)

What Are Filtersets?

Filtersets are advanced configuration options that give you precise control over what data and actions are available through an API endpoint.

How Filtersets Work

When you add a filterset to an endpoint, you specify:

- **Service Name:** Which service (type of data/functionality) can be accessed
  - Example: CaseService, UserService, DocumentService
- **Method Name:** Which specific actions are allowed
  - Example: GetCases, CreateCase, UpdateUser
- **Permission Set:** Template that restricts data fields or actions
  - Example: SIF.GetCasesFilter1 (only basic case info)
- **ResultSet:** Which properties (fields) are exposed in responses
  - Example: Only case number, title, and date

Configuring Filtersets

Step-by-step process:

- **Choose Service Name**
  - Select from dropdown: CaseService, UserService, etc.
  - Limits endpoint to that specific service only
- **Choose Method Name**
  - Select specific method: GetCases, CreateCase, etc.
  - Endpoint can only call this method
- **Select Permission Set**

- Apply filterset template from available templates
- Templates pre-define what data is returned

- **Configure ResultSet**
  - Further limit which fields are exposed
  - Check/uncheck specific properties to hide sensitive data

Benefits of Using Filtersets

1. Enhanced Security

- Only necessary data and actions are exposed
- Reduces risk of unauthorized access
- Prevents third parties from accessing irrelevant information

Example:

Without Filterset:

- Integration can access all case data
- Returns all fields including sensitive information

With Filterset:

- Integration limited to GetCases method only
- Returns only: case number, title, date
- Sensitive fields remain hidden

2. Improved Data Privacy

- Limit which fields are returned
- Ensure personal/confidential data isn't shared unnecessarily
- Supports privacy-by-design principles

Privacy Levels:

Level	What's Exposed	Use Case
Full Access	All fields and methods	Internal trusted systems
Filtered	Specific fields only	Third-party integrations
Minimal	Basic identifiers only	Public-facing APIs

3. Better Performance

- API returns only needed data
- Faster response times
- Reduced load on system
- Dynamic queries instead of large static queries

Performance Comparison:

Configuration	Response Time	Data Size	CPU Load
No filters	2.5 seconds	500 KB	High
With filtersets	0.8 seconds	50 KB	Low

4. Fine-Grained Control

- Decide exactly what each integration can do and see
- Different filtersets for different integrations
- Centralized control and management

Control Matrix Example:

Integration	Service	Methods	Fields Returned
Customer Portal	CaseService	GetCases	Number, Title, Status
Reporting Tool	CaseService	GetCases	All fields
Mobile App	CaseService	GetCases	Number, Title, Date

5. Simplified Maintenance

- Use templates for consistent configurations
- Changes made centrally and applied consistently
- Easy to update and manage
- Reduced configuration errors

Filterset Configuration Example

Scenario: Third-party integration needs basic case information

Configuration:

- Service Name: CaseService
- Method Name: GetCases
- Permission Set: SIF.GetCasesFilter1
- ResultSet:

```
✓ CaseNumber
✓ Title
✓ CreatedDate
✗ ResponsiblePerson (hidden)
✗ InternalNotes (hidden)
✗ SensitiveData (hidden)
```

Result:

- Integration can only call GetCases
- Only receives selected fields

- All other data remains secure

Best Practices for Filtersets

Design:

- ✅ Create filterset templates for common scenarios
- ✅ Document what each filterset allows
- ✅ Use descriptive names (e.g., "PublicCaseInfo", "PartnerAccess")

Security:

- ✅ Default to minimal access, expand as needed
- ✅ Regularly review what data is exposed
- ✅ Test filtersets before deploying

Maintenance:

- ✅ Version your filterset templates
- ✅ Keep documentation updated
- ✅ Audit filterset usage periodically

Best Practices

⚠️ The Best Practices below must be followed when configuring, endpoints. Owners and administrators should verify compliance and provide support where needed.

Authentication Best Practices

✅ DO

- **Use Strong Authentication**

```
Recommended priority:
• Bearer Token (JWT) for user-based access
• AuthKey for service integrations
• Combined authentication for high security
```

- **Leverage Existing Access Controls**

```
• Use Bearer authentication when you want same rules as main system
• Permissions automatically follow user rights
• Simplified administration
```

- **Create Dedicated Service Users**

```
For each integration:
• Create new service user
• Grant minimum necessary permissions
• Use descriptive naming
• Document purpose and owner
```

- **Use Two-Factor Authentication (High-security scenarios)**
  - JWT + AuthKey
  - JWT + AuthKey + ClientID
  - AuthKey + ClientID + ServiceDelegateSecret
- **Store Secrets Securely**
  - ✅ Azure Key Vault
  - ✅ Environment variables
  - ✅ Secure configuration management
  - ❌ Application code
  - ❌ Version control
  - ❌ Public repositories

❌ DON'T

- **Don't Hardcode Secrets**
  - ❌ BAD: const authKey = "8256344C-36DF-414D-B7A4-8ADFA5F5C1E6";
  - ✅ GOOD: const authKey = process.env.API_AUTH_KEY;
- **Don't Share Service Users**
  - Each integration = separate service user
  - Easier tracking and troubleshooting
  - Better security isolation
- **Don't Use Empty AuthKeys for Sensitive Data**
  - ❌ BAD - Sensitive customer data: authKey=""
  - ✅ GOOD: authKey="8256344C-36DF-414D-B7A4-8ADFA5F5C1E6"
- **Don't Disable Security Features**
  - Always validate audience for public APIs
  - Don't allow wildcard CORS origins in production
  - Don't disable rate limiting

Security Best Practices

Permission Management

- **Grant Minimum Necessary Permissions**

```
Ask: What's the minimum this integration needs?
• ❌ Full admin access for read-only dashboard
• ✅ Read permission only for dashboard
```

- **Set Appropriate Permissions per Method**

```
Example endpoint configuration:
• Read: GetCases, GetUsers
```

```
    • Write: CreateCase
```

- **Use Permission Sets**

```
    • Apply filterset templates
    • Restrict data fields returned
    • Limit operations to necessary methods
```

Rate Limiting

- **Set Realistic Limits**
  - Expected request volume
  - Peak usage times
  - System capacity
  - Fair usage policies
- **Monitor and Adjust**
  - Review actual usage patterns
  - Adjust limits based on data
  - Set alerts for limit violations
- **Document Limits**
  - Inform integration developers
  - Include in API documentation
  - Provide guidance on handling rate limits

Client ID Best Practices

- **Use for Tracking and Security** benefits:
  - Track which apps use APIs
  - Audit by application
  - Extra authentication layer
- **Always Send in Headers**
  - ✅ GOOD: Headers: { ClientID: "app-identifier" }
  - ❌ BAD: URL: ?ClientID=app-identifier
- **Generate Unique IDs**
  - One Client ID per application
  - Use GUIDs or similar
  - Don't share between apps

Configuration Management Best Practices

Organization

- **Use Descriptive Names**

    - ❌ BAD: "Endpoint1", "API2"
    - ✅ GOOD: "Customer Portal API", "Mobile App - Case Access"
- **Group Related Endpoints** (Organization structure)
    - Infrastructure (health checks, monitoring)
    - Public APIs (documentation, public data)
    - Partner Integrations (external systems)
    - Internal Integrations (company tools)
- **Document Purpose and Owner** (for each endpoint)
    - PURPOSE: What it's used for
    - OWNER: Which team manages it
    - SECURITY: Why specific security chosen
    - LAST_UPDATED: When last changed

Maintenance Best Practices

Key Rotation

⚠️ Activity for clients/customers.

- **Rotate AuthKeys Regularly**
    - Generate new AuthKey
    - Add to configuration
    - Update clients to use new key

Recommended frequency: Every 6 months

- **Track Key Age**
    - Which keys are in use
    - When they were created
    - When they expire
    - Which integrations use them

Quick Security Checklist

Use this checklist when configuring a new endpoint:

- [ ] Authentication method selected appropriately
- [ ] Bearer for user-based OR AuthKey for service-based
- [ ] Service user created (if using AuthKey)
- [ ] Strong AuthKey generated (GUID)
- [ ] Permissions set to minimum necessary
- [ ] Client ID added (if high security needed)
- [ ] Request limit configured appropriately
- [ ] CORS configured (if web-based integration)
- [ ] Filtersets applied to restrict data/methods

- [ ] Service and method restrictions added
- [ ] Endpoint name is descriptive
- [ ] Purpose and owner documented
- [ ] Tested in development environment
- [ ] Monitoring and logging enabled
- [ ] Security reviewed and approved

Troubleshooting

Common Issues and Solutions

Issue 1: 401 Unauthorized Error

What it means: Authentication failed - the system doesn't recognize who you are.

Common Causes:

- **Wrong or missing AuthKey**

```
Symptom: "Unauthorized" error
Check:
   • Is AuthKey included in request?
   • Does AuthKey match configuration?
   • Is AuthKey sent in header (not URL)?
```

- **Expired JWT Token**

```
Symptom: "Token expired" or "Unauthorized"
Check:
   • Token expiration time
   • System clock synchronization
   • Token refresh logic
```

- **Wrong authentication method**

```
Symptom: Authentication fails even with correct credentials
Check:
  • Is endpoint configured for this auth method?
  • Are you using Bearer when AuthKey is required?
  • Multiple auth methods configured?
```

- **URL pattern doesn't match**

```
Symptom: "Endpoint not found" or "Unauthorized"
Check:
  • Is URL exactly as configured?
  • Case sensitivity (URLs are case-insensitive)
  • Wildcard patterns
```

Solutions:

✅ **Verify AuthKey:**

- Check configuration
- Ensure key is correct

- Send in header: `AuthKey: YOUR-KEY`

✅ Check JWT Token:

- Verify not expired
- Test at https://jwt.io
- Get fresh token

✅ Verify Authentication Method:

- Confirm endpoint configuration
- Check which methods are enabled
- Try each method in order

✅ Check URL:

- Verify exact path
- Check for typos
- Confirm wildcard matching

Issue 2: 403 Forbidden Error

What it means: You're authenticated (system knows who you are) but not authorized (you're not allowed to do this action).

Common Causes:

- **Content validation blocking operation**

```
Symptom: Authentication succeeds, but specific action denied
Reason: Filtersets or permission sets restrict this method
Solution:
  • Contact administrator to add required permissions
  • Check if filterset allows this operation
  • Verify service/method configuration
```

- **Rate limit exceeded**

```
Symptom: "429 Too Many Requests" or "403 Forbidden"
Reason: Too many requests in time window
Solution:
  • Wait before making more requests
  • Implement exponential backoff
  • Request higher rate limit if needed
```

- **Missing required permissions**

```
Symptom: Some actions work, others return 403
Reason: Service user lacks specific permissions
Solution:
  • Check service user permissions
  • Contact administrator to grant access
  • Verify permission requirements
```

- **Client ID mismatch**

```
Symptom: AuthKey valid but still denied
Reason: Client ID doesn't match or is missing
```

```
Solution:
  • Verify Client ID is sent in request
  • Check Client ID matches configuration
  • Send in header, not URL
```

Solutions:

✅ **Check Permissions:**

- Review service user permissions
- Verify filterset configuration
- Confirm method is allowed

✅ **Rate Limiting:**

- Check current usage
- Wait and retry
- Implement proper error handling

✅ **Verify Client ID:**

- Confirm Client ID in request
- Check it matches configuration
- Use header: `ClientID: your-id`

Issue 3: 404 Not Found Error

What it means: The endpoint doesn't exist or can't be found.

Common Causes:

- **URL pattern doesn't match**

```
Symptom: "Not found" error
Reason: URL doesn't match any configured pattern
Solution:
   • Verify URL is correct
   • Check endpoint configuration
   • Confirm wildcard matching rules
```

- **Dynamic endpoints not loaded**

```
Symptom: Endpoint exists in config but returns 404
Reason: Backend service unavailable or not loaded
Solution:
   • Wait 1 minute for automatic refresh
   • Check backend service status
   • Verify endpoint mapper is working
```

- **Endpoint removed**

```
Symptom: Previously working endpoint now 404
Reason: Configuration changed or endpoint deleted
Solution:
  • Contact administrator
  • Check recent configuration changes
  • Verify endpoint still exists
```

Solutions:

✅ Verify URL:

- Check spelling and path
- Confirm exact format
- Test in development first

✅ For Dynamic Endpoints:

- Wait for refresh (1 minute)
- Check backend service health
- Contact administrator

✅ Check Configuration:

- Verify endpoint exists
- Check if recently changed
- Review configuration logs

Issue 4: CORS Errors (Browser Only)

What it means: Browser security is blocking the cross-origin request.

Common Symptoms:

```
Browser console error: "Access to fetch at 'https://api.company.com/v2/api' from origin
'https://app.example.com' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is
present."
```

Common Causes:

- **Origin not allowed**

```
Symptom: CORS error in browser
Reason: Your website domain isn't in allowed origins list
Solution:
  • Contact administrator to add your domain
  • Verify exact domain (including https://)
  • Check for typos in domain name
```

- **CORS not enabled**

```
Symptom: No CORS headers in response
Reason: CORS configuration not enabled for endpoint
Solution:
  • Contact administrator to enable CORS
  • Verify CORS is needed (browser-based only)
  • Check endpoint configuration
```

- **Wrong headers or methods**

```
Symptom: Preflight request fails
Reason: Trying to send headers/methods not in allowed list
 Solution:
  • Check which headers are allowed
```

```
    • Verify methods (GET, POST, etc.) are permitted
    • Contact administrator to update CORS config
```

Solutions:

✅ Add Your Domain:

Contact admin to add to allowed origins:

- `https://your-app.company.com`
- Include subdomain exactly
- No trailing slashes

✅ Enable CORS:

Required settings:

- Enable CORS: true
- Allowed Origins: [your domain]
- Allowed Methods: GET, POST, OPTIONS
- Allowed Headers: authorization, content-type
- Allow Credentials: true (if using auth)

✅ Test CORS:

- Use browser dev tools
- Check preflight OPTIONS request
- Verify response headers

Issue 5: Configuration Changes Not Applied

What it means: You made configuration changes but they're not taking effect.

For Static Configuration:

```
Symptom: Changes in config file not reflected
Reason: Service not restarted
Solution:
  • Save configuration file
  • Restart the AuthProxy service
  • Wait for service to fully start
  • Test endpoint again
```

Note: Static config requires restart!

For Dynamic Configuration:

```
Symptom: Changes in admin UI not reflected
Reason: Refresh interval hasn't passed yet
Solution:
  • Wait up to 1 minute for automatic refresh
  • Verify backend service is accessible
  • Check service logs for errors
  • Force refresh if needed (restart service)
```

Default refresh: Every 1 minute (configurable via the **SI.Biz.AuthProxy.EndpointCheckInterval** app setting)

Verification Steps:

- [ ] Confirm change was saved
- [ ] Check configuration version
- [ ] Verify no syntax errors
- [ ] Review service logs
- [ ] Test with diagnostic mode enabled
- [ ] Contact administrator if persists

Issue 6: Token Validation Fails

What it means: JWT token is present but fails validation.

Common Causes:

- **Wrong metadata URI**

```
   Symptom: "Unable to validate token"
   Reason: Token issuer doesn't match metadata URI
   Solution:
     • Verify metadata URI is correct
     • Check token issuer claim
     • Test metadata endpoint
```

- **Token signature invalid**

```
   Symptom: "Invalid signature"
   Reason: Token signed with wrong key or tampered
   Solution:
     • Get fresh token from identity provider
     • Verify token hasn't been modified
     • Check signing key configuration
```

- **Wrong OAuth scope (most common cause of IDX10511)**

```
   Symptom: "IDX10511: Signature validation failed"
   Reason: Token was requested with scope targeting the wrong resource (e.g., graph.microsoft.com instead of the
360 app registration). The token is signed with different keys than what the metadata URI provides.
   Solution:
     • Check the scope used in the token request — must be api://<client-id>/.default
     • Decode token at jwt.io and verify the aud claim matches your app registration, not graph.microsoft.com
```

- **Signing keys not loaded (Configuration: 0 keys)**

```
   Symptom: "IDX10511: Signature validation failed" with "Number of keys in Configuration: '0'"
   Reason: The AuthProxy hasn't loaded signing keys from the endpoint's metadata URI yet.
   Solution:
     • Verify metadata URI returns valid JSON in a browser
     • Wait up to 1 minute for automatic config refresh (controlled by SI.Biz.AuthProxy.EndpointCheckInterval app
setting)
     • Force refresh by restarting the AuthProxy service if needed
```

- **Claims missing**

```
   Symptom: "User claim not found"
   Reason: Token doesn't contain expected user claim
   Solution:
```

- Check which claims are in token
- Verify user claim configuration
- Update token request scope

Debugging JWT Issues:

- **Decode token at https://jwt.io**
  - Check expiration time
  - Verify issuer
  - Confirm user claims present
- **Test metadata endpoint:**

```
curl https://login.microsoftonline.com/common/v2.0/.well-known/openid-configuration
```

- **Verify signing keys:**

```
curl https://login.microsoftonline.com/common/discovery/v2.0/keys
```

- **Check configuration:**
  - Metadata URI correct?
  - User claim matches token?
  - Token not expired?

Diagnostic Tips

Test Authentication

Combined Daemon (client_credentials):

Step 1 — Get app token:

```
curl -X POST "https://login.microsoftonline.com/<tenant-id>/oauth2/v2.0/token" \
  -d "grant_type=client_credentials" \
  -d "client_id=<your-client-id>" \
  -d "client_secret=<your-secret>" \
  -d "scope=api://<your-client-id>/.default"
```

Step 2 — Call SIF API:

```
curl -X POST "https://<environment>/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/CaseService/GetCases" \
  -H "Authorization: Bearer <token-from-step-1>" \
  -H "authkey: <value-from-360-admin>" \
  -H "ClientID: <your-client-id>"
```

Bearer (user token, interactive login):

- Use Postman with OAuth 2.0 Authorization Code grant type
- Scope: `api://<client-id>/.default`
- Callback URL: `https://oauth.pstmn.io/v1/callback` (must be registered as redirect URI in Azure app registration)
- The user you log in with must exist in Public 360 with a matching identity (UPN or email)

Check response:

- 200 OK = Success
- 401 = Authentication failed (missing or wrong headers)
- 403 = Token validation failed (wrong scope, issuer mismatch, or keys not loaded)
- 404 = Endpoint not found

Get Help Information

When reporting issues to support, provide:

- [ ] Error message (exact text)
- [ ] When it occurred (date/time)
- [ ] Endpoint URL you're accessing
- [ ] Authentication method you're using
- [ ] Request headers (remove sensitive data)
- [ ] Response status code
- [ ] Steps to reproduce
- [ ] Expected vs. actual behavior
- [ ] Environment (dev/test/production)

Error Code Quick Reference

Code	Meaning	Common Fix
401	Unauthorized - Authentication failed	Check AuthKey or token
403	Forbidden - Not authorized	Verify permissions
403 (IDX10511)	Signature validation failed	Check OAuth scope is `api://<client-id>/.default`, not `graph.microsoft.com`
404	Not Found - Endpoint doesn't exist	Check URL and configuration
429	Too Many Requests - Rate limit	Wait and retry
500	Server Error - Backend issue	Contact administrator

Summary

Key Takeaways

1. Choose the Right Authentication

Decision Matrix:

- User-based access (permissions follow user): → Use Bearer Token (JWT)
- Service-to-service automation: → Use AuthKey with dedicated service user
- High security external integration: → Use Combined (JWT + AuthKey + Client ID)
- Internal applications: → Use Windows Authentication

- Legacy systems: → Use Basic Authentication (HTTPS only!)

2. Layer Your Security

Security Layers (add as needed):

Basic Security:

- [ ] Authentication method (Bearer or AuthKey)

Enhanced Security:

- [ ] + Client ID (two-factor)
- [ ] + Rate limiting

High Security:

- [ ] + Service Delegate Secret
- [ ] + Filtersets (restrict data/methods)
- [ ] + CORS (web apps)
- [ ] + Permission sets

3. Configure Properly

Configuration Checklist:

Setup:

- [ ] Descriptive endpoint name
- [ ] Correct URL pattern
- [ ] Appropriate authentication method
- [ ] Dedicated service user (if AuthKey)

Security:

- [ ] Strong AuthKey (GUID)
- [ ] Minimum necessary permissions
- [ ] Client ID (if needed)
- [ ] Rate limits set

Advanced:

- [ ] Filtersets configured
- [ ] CORS settings (if web app)
- [ ] Service/method restrictions

Testing:

- [ ] Tested in development
- [ ] Verified authentication
- [ ] Checked permissions

- [ ] Monitored logs

4. Maintain Security

Regular Maintenance:

Monthly:

- [ ] Review endpoint usage
- [ ] Check for anomalies
- [ ] Verify rate limits
- [ ] Audit failed authentications

Quarterly:

- [ ] Rotate AuthKeys
- [ ] Review permissions
- [ ] Audit service users
- [ ] Update documentation

Annually:

- [ ] Full security audit
- [ ] Review all endpoints
- [ ] Update security policies
- [ ] Train team on best practices

5. Monitor and Improve

Continuous Improvement:

Monitor:

- Request volumes
- Error rates
- Authentication failures
- Rate limit hits
- Performance metrics

Analyze:

- Usage patterns
- Security incidents
- Performance bottlenecks
- Integration health

Optimize:

- Adjust rate limits
- Refine permissions
- Update filtersets

- Improve documentation

Quick Reference Cards

Authentication Methods Quick Guide

Need	Use	Example
User-based access	Bearer Token (JWT)	Mobile app, web app
Service automation	AuthKey	Scheduled jobs, integrations
High security	JWT + AuthKey	Partner integration

Security Quick Guide

Feature	Purpose	When to Use
AuthKey	Identify integration	Always for service-based
Client ID	Identify application	High security needs
Rate Limiting	Prevent abuse	All public/external APIs
CORS	Control browser access	Web-based integrations
Filtersets	Restrict data/methods	Third-party access

Troubleshooting Quick Guide

Error	Likely Cause	Quick Fix
401	Wrong AuthKey/token	Verify credentials
403	No permission	Check service user rights
404	Wrong URL	Verify endpoint config
429	Too many requests	Wait and retry
CORS	Domain not allowed	Add domain to CORS config

Additional Resources

Getting Help

For Configuration Support:

- Contact your system administrator
- Check internal documentation
- Review endpoint configuration portal

For Technical Issues:

- Enable diagnostic logging
- Check Windows Event Viewer
- Review error messages in detail

When Reporting Issues:

Provide complete information:

1. Error message (exact text)
2. Timestamp
3. Endpoint URL
4. Authentication method
5. Request details (headers, body)
6. Expected vs. actual behavior
7. Steps to reproduce
8. Environment (dev/test/prod)

Best Practice Templates

Service User Naming: Format: SvcUser_[Integration]_[Purpose] Examples:

- SvcUser_CustomerPortal_ReadOnly
- SvcUser_ReportingAPI_FullAccess
- SvcUser_MobileApp_Cases

Endpoint Naming: Format: [System][Purpose][Type] Examples:

- CustomerPortal_CaseAccess_Public
- Mobile_UserData_Internal
- PartnerAPI_DocumentSync_External

Documentation Template:

- Endpoint: [Name]
- Purpose: [What it's for]
- Owner: [Team/Person]
- Created: [Date]
- Authentication: [Method]
- Permissions: [List]
- Rate Limit: [Number/minute]
- CORS: [Enabled/Disabled]
- Service User: [Name]
- Notes: [Special considerations]

Miscellaneous

How an endpoint is selected when a Bearer token is sent together with an AuthKey and ClientID

⚠️ This describes the runtime selection logic implemented in the AuthProxy's flow. Useful for administrators, integrators and troubleshooting.

Summary (high-level)

- When a request contains a Bearer (JWT) token the authproxy (API gateway) prefers endpoints that explicitly match the token + integration identifiers. It first tries endpoints that are configured to accept both the ClientID and the AuthKey, then endpoints configured for ClientID only, then endpoints configured for AuthKey only. If none of those authorize the request, the proxy will try any remaining endpoints as a fallback. The first endpoint that fully authorizes the request is returned.

Selection process details

- When a request arrives with a user token (Bearer/JWT) plus an integration key (AuthKey) and an app id (ClientID), the gateway tries to pick the most specific and secure match first.
- It checks for endpoints that explicitly expect both the ClientID and the AuthKey. If one of those accepts the request, that endpoint is used.
- If none match both, it next tries endpoints that match the ClientID alone, then endpoints that match the AuthKey alone.
- If still nothing matches, it falls back to any remaining endpoints it can find and tries those.
- The first endpoint that fully accepts the request is used. If none accept it, the request is denied.

Why this matters

- This order ensures the gateway prefers the most specific configuration (both app + key), reducing the chance an integration accidentally uses the wrong endpoint.
- It also enforces the intended security checks (token validation, key and client checks) before allowing access.

Simple example

- Request headers: Authorization: Bearer <token>, AuthKey: ABC, ClientID: app-1
- Runtime: the gateway looks for endpoints configured for (app-1 + ABC) → then (app-1) → then (ABC) → then any others.

Practical tips for non-technical readers

- Give each integration a unique ClientID and AuthKey so the gateway finds the right endpoint.
- Configure each endpoint for only what it needs (least privilege) so even if credentials match, the integration only gets allowed access.
- If something fails, support teams will look for logs that show the gateway checking endpoints in that order and which check failed.

Short takeaway

- The gateway prefers most-specific matches (ClientID+AuthKey) first, then less specific matches; unique identifiers and minimal permissions prevent accidental or overly broad access.

Security note

- Ensure endpoints are configured with the least privilege (filtersets and roles) and that ClientID/AuthKey pairs are unique per integration to avoid unintended matches.

Changelog

- **Document Version:** 4.0 (Complete User Guide)

- **Last Updated:** 2026-05-11
- **Maintained By:** Core-integration Team

Feedback Welcome Help us improve this guide! If you have suggestions, found errors, or have questions, please contact the Core-integration team.

Retrieved from "https://wiki.software-innovation.com/index.php?title=Endpoint_configuration&oldid=59744"

This page was last edited on 4 June 2026, at 08:25.